

NovaSMS

Résultats, Bilan, Limites & Perspectives

Résultats mesurés en exécutant réellement tests, lint et builds · Bilan chronologique · Dette technique · Limites · Perspectives

Stage Sankofa Lab · Romuald KO · Document régénéré depuis le code réel du dépôt
Plateforme SaaS B2B Messagerie Multi-Canal — NestJS · PostgreSQL · Redis · React 19

Table des matières

1	Méthodologie de mesure des résultats
2	Résultats — tests unitaires backend
3	Résultats — couverture de code backend
4	Résultats — tests unitaires frontend
5	Résultats — tests end-to-end (Playwright)
6	Résultats — qualité statique (lint, typage, build)
7	Anomalie critique détectée — build frontend cassé
8	Intégration continue (CI/CD)
9	Bilan chronologique du développement
10	Objectifs atteints vs objectifs non atteints
11	Dette technique consolidée
12	Limites du projet
13	Perspectives d'évolution
14	Conclusion générale

1 — Méthodologie de mesure des résultats

Contrairement à une présentation de résultats reconstituée a posteriori à partir de notes ou de souvenirs, chaque chiffre de ce document a été obtenu en exécutant réellement, le 20 juillet 2026, les commandes suivantes sur le code du dépôt (branche feat/automations-bullmq-tests, environnement Docker local avec PostgreSQL 15 et Redis 7 actifs) :

- **npm run test (backend)** !' suite Jest complète, 67 fichiers de spécification
- **npx jest --coverage (backend)** !' mesure de couverture de code réelle (lignes, fonctions, branches)
- **npx vitest run (frontend)** !' suite de tests unitaires
- **npx playwright test (frontend + backend démarrés automatiquement)** !' 10 fichiers, ~45 scénarios end-to-end
- **npx eslint (backend et frontend)** !' analyse statique
- **npx tsc --noEmit / npm run build (backend et frontend)** !' vérification de typage et compilation de production

Cette démarche garantit que les chiffres présentés dans ce document reflètent l'état réel du code au moment de la rédaction, et non un objectif ou une estimation.

613

tests unitaires exécutés

67 fichiers de spécification (*.spec.ts)

608 / 613

tests passants (99,2 %)

2 échecs, 3 tests ignorés (skipped)

Détail de l'exécution (jest --runInBand --forceExit) : 66 suites passantes sur 67, durée totale 7 à 12 secondes selon la charge machine.

2.1 Échec identifié

' 2 tests en échec — templates/templates.controller.spec.ts

TemplateController: Cannot read properties of undefined (reading 'accountId') dans TemplateController.create et TemplateController.findById. Cause : le contrôleur lit req.accountId injecté en production par le TemplateInterceptor global, mais le mock de requête utilisé dans ce fichier de test ne définit pas cette propriété. Il s'agit d'un défaut du test (test incomplet), pas d'un bug du code de production — mais il signale que ce contrôleur n'a plus été testé depuis l'introduction du TemplateInterceptor, et doit être corrigé avant la maintenance pour ne pas laisser de rouge dans le CI.

2.2 Répartition des tests par domaine (fichiers *.spec.ts)

Domaine	Fichiers spec	Volume de tests (ordre de grandeur)
providers (SMS/Email/WhatsApp/paiement/stockane)	~12 fichiers	100+ tests — le domaine le plus testé
auth (authentification, JWT, 2FA)	6 fichiers	~76 tests
contacts + segments (CRM, import, RGPD)	9 fichiers	~103 tests
campaigns (création, cycle de vie, A/B, upload)	8 fichiers	~94 tests
webhooks (réception + abonnements)	3 fichiers	~40 tests
automations + queues (workflows, BullMQ)	10 fichiers	~59 tests
mobile-money + transactions (paiement)	4 fichiers	~37 tests
account, templates, analytics, audit-logs, common	—	~62 tests

Constat : la répartition des tests suit fidèlement la criticité métier des modules — les flux à fort enjeu (envoi de messages, fournisseurs externes, authentification, paiement) concentrent la majorité de l'effort de test, ce qui est cohérent avec une démarche de test orientée risque plutôt qu'une couverture uniforme.

2.3 Modules sans fichier de test dédié

api-keys (guards et interceptors non couverts directement) et public-api. Ce point est repris au chapitre 12 (Limites).

3 — Résultats — couverture de code backend

Mesure obtenue via `npx jest --coverage`, calculée à partir du fichier `lcov.info` généré (agrégation de toutes les lignes/fonctions/branches instrumentées du dossier `src`).

Métrique	Couvert / Total	Pourcentage
Lignes de code	3 064 / 4 360	70,3 %
Fonctions	474 / 667	71,1 %
Branches (conditions)	1 673 / 3 246	51,5 %

🔍 Interprétation

La couverture de lignes/fonctions (~70-71 %) est un niveau solide pour un backend de cette taille développé en solo sur 12 semaines. La couverture de branches, plus faible (51,5 %), est un indicateur classique de tests qui valident le chemin nominal (cas de succès) mais couvrent moins systématiquement les branches d'erreur et les cas limites (ex. échecs partiels de providers, valeurs limites de validation). Aucun seuil minimal de couverture (`coverageThreshold`) n'est configuré dans `jest` — la CI ne peut donc pas bloquer une régression de couverture, contrairement au nombre de tests en échec.

4 — Résultats — tests unitaires frontend

8

10 / 10

tests passants (100 %)

2 fichiers, exécution en 137 ms

Vitest 4.1.8

environnement node

Aucun test de composant React (pas de @testing-library/react)

Les deux seuls fichiers de test unitaire du frontend sont `src/lib/blocksToHtml.test.ts` (conversion des blocs de l'éditeur email en HTML — interpolation de variables, lien de désabonnement RGPD, rendu de bouton) et `src/lib/validation.test.ts`. Ce dernier teste un schéma zod (`RegisterSchema` de `src/lib/validation.ts`) qui n'est en réalité jamais utilisé par l'application — le formulaire d'inscription utilise un schéma défini localement et en double dans `RegisterForm.tsx` (voir Doc 3, chapitre 14). Ce test valide donc un code mort plutôt que le comportement réel de production, ce qui limite fortement la valeur de couverture apportée par ce fichier malgré son succès.

5 — Résultats — tests end-to-end (Playwright)

45 / 45

scénarios passants (100 %)

10 fichiers, exécution complète en 43,9 s

Backend + Frontend

démarrés automatiquement

webServer Playwright, environnement réel (pas de mock)

Fichier	Scénarios	Couvre
auth.spec.ts	7	Login, erreurs, mot de passe oublié, route protégée
navigation.spec.ts	8	Smoke test de toutes les pages protégées
contacts.spec.ts	7	Liste, recherche, détail, sélection, bouton nouveau contact
contact-detail.spec.ts	3	Fiche contact seedée, email/téléphone affichés, contact inconnu
account.spec.ts	7	Sécurité, paramètres, équipe, logs, intégrations, rechargement
segments.spec.ts	4	Builder de segment (via la page Contacts réelle)
campaigns.spec.ts	3	Bouton de création, wizard, liste vide
automations.spec.ts	3	Page automations sans crash JS
analytics.spec.ts	3	Page charge, filtres de période

Ø=Û Nature des tests — à assumer explicitement

La majorité de ces scénarios sont des smoke tests (la page se charge sans erreur JavaScript, un élément clé est visible) plutôt que des tests fonctionnels profonds. Aucun scénario ne couvre : le parcours de paiement Mobile Money jusqu'à confirmation, l'envoi effectif d'une campagne, ou l'activation de la 2FA. Ce niveau de test suffit à garantir la non-régression de navigation, mais pas la validation métier de bout-en-bout des flux financiers — point à traiter en priorité dans une prochaine itération (voir chapitre 13).

6 — Résultats — qualité statique (lint, typage, build)

Vérification	Backend	Frontend
ESLint — erreurs	1 erreur	0 erreur
ESLint — avertissements	17 avertissements	3 avertissements
tsc --noEmit (vérification CI)	0 erreur	0 erreur (voir chapitre 7 — résultat trompeur)
Build de production	nest build !' succès	npm run build !' ÉCHEC (voir chapitre 7)

6.1 Détail de l'erreur ESLint backend

' **common/billing.util.ts:46 — no-control-regex**
Unexpected control character in regular expression: 000. Une expression régulière du calcul de coût SMS contient un caractère de contrôle littéral, probablement destiné à détecter l'encodage Unicode des messages. Par convention, on attend en tout des tests du module `assert`, mais à changer — un caractère de contrôle non échappé dans une regex est un signal de qualité que la CI devrait bloquer.

6.2 Avertissements notables (non bloquants)

- **Backend** !' variables/imports déclarés non utilisés (`countSmsParts`, `getUnitPrice` dans `campaigns.service.ts`, `HttpCode` dans `track.controller.ts`), usage de `@ts-ignore/@ts-nocheck` déconseillé, promesses non attendues dans `segments.controller.spec.ts` (`no-floating-promises`)
- **Frontend** !' 3 avertissements `react-hooks/exhaustive-deps` (dépendances manquantes dans des `useEffect` de `CampaignAudienceStep`, `CampaignScheduleStep` et `Team.tsx`) — risque de données obsolètes affichées si la dépendance manquante change sans redéclenchement de l'effet

Ø=ÞÑ Constat

La commande officielle de build du frontend (non run build, exécutée `"tsc -b && vite build"`) échoue actuellement sur la branche `feat(contacts,automations)` build testé avec 2 erreurs de type dans `src/pages/Automations.tsx` (ligne 554 et 570) : le littéral `'birthday'` n'est pas assignable au type `AutomationTrigger`.

7.1 Cause racine

Le type `AutomationTrigger` est défini dans `src/api/automations.ts` et énumère les déclencheurs valides côté frontend. Le commit du 15 juillet 2026 ("`feat(contacts,automations): bouton suppression visible, anniversaire...`") a introduit un déclencheur "anniversaire" (`birthday`) dans la logique d'`Automations.tsx` sans mettre à jour cette définition de type. Comme signalé au Doc 3 (chapitre 11), l'absence de types partagés générés automatiquement entre backend et frontend rend ce genre de désynchronisation possible et silencieuse jusqu'au build.

7.2 Pourquoi la CI ne l'a pas détecté

&™ Explication technique

Le job CI "`lint-and-typecheck`" exécute `npx tsc --noEmit` (sans l'option `-b`) sur `apps/frontend`. Or, `apps/frontend/tsconfig.json` est un fichier de composition de projet ("`files`": [], avec des "`references`" vers `tsconfig.app.json` et `tsconfig.node.json`) : un `tsc --noEmit` exécuté sans `-b` sur ce fichier ne compile de fait AUCUN fichier — la vérification réussit toujours trivialement, qu'il y ait ou non des erreurs de type dans le code. Seule la commande réelle de build (`tsc -b && vite build`, utilisée en local mais pas en CI) exécute la vérification de type complète — et c'est elle qui échoue.

7.3 Portée et action recommandée

Ce défaut est double : (1) un bug de type réel actuellement présent sur la branche, qui empêcherait un déploiement de production tant qu'il n'est pas corrigé (ajouter `'birthday'` à `AutomationTrigger`), et (2) un défaut de configuration CI qui masque ce type d'erreur depuis sa mise en place — le job de typecheck ne vérifie en réalité rien côté frontend. La correction recommandée est double : ajouter le littéral manquant au type, et remplacer `npx tsc --noEmit` par `npx tsc -b --noEmit` (ou exécuter directement `npm run build`) dans le workflow CI pour que ce type de régression soit bloqué automatiquement à l'avenir.

8 — Intégration continue (CI/CD)

10

.github/workflows/ci.yml définit un pipeline "NovaSMS CI" à 3 jobs séquentiels, déclenché sur push/PR vers main et develop.

Job	Contenu	Constat
lint-and-typecheck	npm ci, prisma generate, ESLint, tsc --noEmit (front+back)	Le typecheck frontend est un faux-positif (chapitre 7)
test-backend	Services Docker PostgreSQL 15 + Redis 7, npm run test	Exécute uniquement les tests unitaires
build	Build frontend (Vite) puis backend (nest build)	Ce job échouerait réellement s'il était exécuté aujourd'hui (chapitre 7)

Ni les tests end-to-end Playwright, ni la couverture de code, ni un déploiement automatique ne sont intégrés à la CI actuelle. Le pipeline valide donc la qualité du code (lint, typage nominal, tests unitaires) mais pas la non-régression fonctionnelle bout-en-bout à chaque changement.

9 — Bilan chronologique du développement

11

Reconstitution basée sur l'historique Git réel du dépôt (81 commits, du 23 avril au 15 juillet 2026, semaines ISO).

Période	Semaine ISO	Commits	Contenu principal observé
20–26 avr. 2026	S17	3	Initialisation monorepo, outillage (ESLint/Prettier/Husky), Docker Compose
4–17 mai 2026	S19–S20	8	Authentification, 2FA, onboarding
18–31 mai 2026	S21–S22	14	Gestion de compte, contacts, premiers imports
1–14 juin 2026	S23–S24	15	Campagnes — wizard, éditeurs SMS/Email, A/B testing
15–28 juin 2026	S25–S26	19	Automatisations, BullMQ, tests e2e, analytics
29 juin–12 juil. 2026	S27–S28	14	Stabilisation, API publique, webhooks, correctifs
13–19 juil. 2026	S29	8	Validation téléphone mondiale, UX contacts/automatisations

Le pic d'activité (semaine S25, 16 commits) correspond au développement du module automatisations avec ses tests BullMQ dédiés — cohérent avec la complexité technique de ce module (files d'attente, écoute d'événements, workers) identifiée dans le Doc 2.

10 — Objectifs atteints vs objectifs non atteints

12

10.1 Objectifs fonctionnels (Doc 1, chapitre 5.1)

Objectif	Statut	Constat
OF-1 Auth & équipe multi-rôle	Atteint	76 tests unitaires, 7 scénarios e2e passants
OF-2 Contacts, import, segments, RGPD	Atteint (avec réserve)	Fonctionnel ; UI segments dédiée non branchée au routing (Doc 3 §6.3)
OF-3 Campagnes SMS/Email, A/B	Atteint (avec réserve)	Fonctionnel ; canal WhatsApp absent du wizard de campagne
OF-4 Automatisations événementielles	Atteint	Build actuellement cassé sur cette branche (chapitre 7) — à corriger
OF-5 Paiement Mobile Money + carte	Atteint (en simulation)	Intégrations réelles non testées en conditions de production
OF-6 Tableau de bord analytique	Atteint	Fonctionnel, sans librairie de graphiques malgré sa présence en dépendance
OF-7 API publique pour intégration tierce	Atteint	v1 fonctionnelle ; module api-keys sans tests unitaires dédiés

10.2 Objectifs techniques (Doc 1, chapitre 5.2)

Objectif	Statut	Constat
OT-1 Isolation multi-tenant	Atteint	TenantInterceptor + accountId en cascade sur 26 modèles
OT-2 Traitement asynchrone (BullMQ)	Atteint	5 files actives ; une incohérence @nestjs/bull vs bullmq (Doc 2 §5.3)
OT-3 Abstraction fournisseurs (Factory)	Atteint	6 familles de providers ; le provider Push n'est pas raccordé (Doc 2 §10.4)
OT-4 Sécurité applicative	Atteint	JWT, 2FA, blacklist, RBAC, HMAC webhooks, rate-limiting
OT-5 Couverture de tests + CI	Partiellement atteint	608/613 tests backend, 45/45 e2e ; CI ne détecte pas les erreurs de type frontend (chapitre 7)

11 — Dette technique consolidée

13

Synthèse de tous les points de dette technique identifiés dans les Docs 2 et 3, consolidés ici pour prioriser une éventuelle feuille de route de stabilisation post-stage.

Priorité	Élément	Impact
Haute	Build frontend cassé (AutomationTrigger)	Bloque tout déploiement de production en l'état
Haute	Job CI typecheck frontend non fonctionnel	Aucune protection réelle contre les erreurs de type
Moyenne	2 tests templates.controller.spec.ts en échec	Pollue le signal de la CI, cache d'éventuelles vraies régressions
Moyenne	@nestjs/bull vs @nestjs/bullmq incohérent	Dépendance non déclarée utilisée en production (segments)
Moyenne	Deux implémentations mortes (campagnes, segments)	Confusion pour toute nouvelle personne reprenant le code
Basse	Dépendances jamais utilisées (recharts, @dnd-kit...)	Poids de bundle et de maintenance inutile
Basse	Double stratégie de validation (class-validator / zod)	Incohérence de style, pas de risque fonctionnel direct
Basse	i18n initialisée mais non déployée (2 clés sur toute l'app)	Fonctionnalité commencée puis non poursuivie

- **Couverture de branches perfectible (51,5 % côté backend)** !' les chemins d'erreur et cas limites sont moins testés que les chemins nominaux
- **Tests e2e majoritairement de type smoke test** !' aucun scénario ne valide un paiement Mobile Money jusqu'à confirmation, ni un envoi de campagne réel jusqu'à réception
- **Absence de tests de charge/performance** !' aucun outil de test de charge (k6, Artillery...) n'est présent dans le dépôt ; l'objectif "import de 50 000 lignes en moins de 60 secondes" mentionné dans le code (Doc 2 §5.2) n'est donc pas vérifié par une mesure automatisée reproductible
- **Pas de tests de composants frontend** !' aucune dépendance de test de rendu React (@testing-library/react) n'est installée
- **Modules api-keys et public-api sans suite de tests dédiée** !' zone de risque sur l'API exposée publiquement aux clients
- **Absence de génération de types partagés backend/frontend** !' source directe du bug de build identifié au chapitre 7
- **Un seul développeur sur l'ensemble du projet** !' pas de revue de code par les pairs pendant le développement, ce qui explique en partie l'accumulation de code mort documentée

13.1 Correctifs à court terme (avant mise en production)

- Corriger le type AutomationTrigger et remplacer tsc --noEmit par tsc -b --noEmit dans la CI
- Corriger les 2 tests templates.controller.spec.ts et l'erreur ESLint no-control-regex
- Unifier @nestjs/bull vers @nestjs/bullmq dans segment.recalculation.service.ts
- Supprimer le code mort identifié (Doc 3 §14) pour réduire la surface de confusion

13.2 Évolutions produit envisageables

- Raccorder le canal WhatsApp au wizard de création de campagne (le provider existe déjà côté backend)
- Raccorder le module de notifications push (Firebase) déjà implémenté et testé côté backend
- Introduire une couche de cache de données côté frontend (React Query ou équivalent) pour mutualiser les appels /analytics/overview dupliqués entre Dashboard et Analytics
- Construire un design system de composants réutilisables pour unifier l'expérience visuelle (actuellement disparate entre pages CSS custom et pages Tailwind "Material 3")
- Ajouter des tests e2e fonctionnels profonds sur les flux financiers (Mobile Money, carte bancaire) et l'envoi effectif de campagne
- Générer un package de types partagés entre Prisma (backend) et le frontend pour éliminer la classe de bugs illustrée au chapitre 7

14 — Conclusion générale

10

Ce projet de stage a permis de livrer une plateforme SaaS B2B fonctionnelle couvrant l'intégralité de la chaîne métier visée par la problématique initiale (Doc 1) : de l'inscription d'un compte à l'envoi d'une campagne multicanale suivie analytiquement, en passant par la gestion de contacts, l'automatisation et le paiement local. Les résultats mesurés dans ce document — 608 tests unitaires backend passants sur 613, 45 scénarios end-to-end passants sur 45, une couverture de code backend de 70,3 % en lignes — attestent d'un niveau de qualité substantiel pour un développement mené par une seule personne sur environ 12 semaines.

La démarche adoptée pour ce document — exécuter réellement chaque suite de tests plutôt que d'en estimer les résultats — a également mis en évidence une anomalie de build actuellement bloquante sur la branche de travail (chapitre 7), ainsi qu'un inventaire assumé de dette technique (chapitres 11-12). Loin d'affaiblir le bilan du stage, la capacité à détecter, documenter et prioriser ces points constitue en elle-même une compétence d'ingénierie logicielle démontrée, et fournit une feuille de route claire et actionnable pour la suite du projet au-delà de la période de stage.

' Chiffres clés à retenir pour la soutenance

613 tests unitaires backend (608 passants) · 10 tests unitaires frontend (10 passants)

45 scénarios end-to-end passants sur 45 (100 %) · Couverture backend : 70,3 % lignes / 51,5 % branches

26 modèles de données, 38 migrations, ~20 modules backend, 81 commits sur 12 semaines

1 anomalie critique de build identifiée et documentée avec cause racine et correctif proposé